

BLG470E - Functional Programming

HW2

Dr. Yusuf H. Şahin

Answer the following questions using Haskell. You may define helper functions when necessary.

Q1 - Menger Sponge

A *Menger sponge* is a three-dimensional fractal constructed recursively from a cube. At each iteration, every cube is processed as follows:

1. Divide the cube into a $3 \times 3 \times 3$ grid of 27 equally sized subcubes.
2. Remove the subcube at the exact center.
3. Remove the six subcubes located at the centers of the faces.
4. Apply the same procedure recursively to each of the remaining 20 subcubes.

Use the provided three-dimensional model representation:

```
type Point = (Float, Float, Float)
type Triangle = (Point, Point, Point)
type Shape = [Triangle]
```

The following function is already provided. It creates a cube centered at the given point with the specified side length. Each cube is represented using 12 triangles.

```
cubeAt :: Point -> Float -> Shape
```

Write the following function:

```
mengerSponge :: Int -> Shape
```

The function must generate a Menger sponge centered at (0,0,0). The initial cube must have side length 60. The parameter represents the number of fractal iterations.

You may define an internal helper function with the following signature:

```
menger :: Int -> Point -> Float -> Shape
```

For a cube centered at (x,y,z) with side length s, the centers of its 27 possible subcubes can be expressed as

$$\left(x + i\frac{s}{3}, y + j\frac{s}{3}, z + k\frac{s}{3}\right), \quad i, j, k \in \{-1, 0, 1\}.$$

A subcube must be removed whenever at least two of i, j, and k are equal to zero. All other subcubes must be retained.

Your implementation must satisfy the following conditions:

```
length (mengerSponge 0) == 12
length (mengerSponge 1) == 240
length (mengerSponge 2) == 4800
```

The solution must be recursive. Do not write separately hard-coded definitions for different iteration counts.

The generated shape can be exported using the provided function:

```
writeObjModel (mengerSponge 3) "menger.stl"
```

After generating the 3D Shape, you can use CloudCompare to visualize it.

Q2 - Conway's Game of Life

Conway's Game of Life is played on an infinite two-dimensional grid. Each cell is either *alive* or *dead*. The next generation is determined according to the following rules:

1. A live cell with two or three live neighbours remains alive.
2. A dead cell with exactly three live neighbours becomes alive.
3. All other cells become or remain dead.

A cell is represented by its Cartesian coordinates. A board contains only the coordinates of the currently live cells.

```
type Cell = (Int, Int)
type Board = [Cell]
```

The following functions are provided:

```
import Data.List (nub, sort)

neighbours :: Cell -> [Cell]
neighbours (x, y) =
  [ (x + dx, y + dy)
  | dx <- [-1, 0, 1]
  , dy <- [-1, 0, 1]
  , (dx, dy) /= (0, 0)
  ]

liveNeighbours :: Board -> Cell -> Int
liveNeighbours board cell =
  length [p | p <- neighbours cell, p `elem` board]

candidates :: Board -> [Cell]
candidates board =
  nub (board ++ concatMap neighbours board)
```

1. Complete the missing Boolean expression in `nextGeneration` so that it returns the live cells of the next generation.

```
nextGeneration :: Board -> Board
nextGeneration board =
  [ cell
  | cell <- candidates board
  , let n = liveNeighbours board cell
  , -----
  ]
```

2. Complete the recursive function that calculates the board after n generations.

```
runLife :: Int -> Board -> Board
runLife 0 board = board
runLife n board =
    -----
```

3. Consider the following initial board, called a *glider*:

```
glider :: Board
glider =
    [ (1,0)
    , (2,1)
    , (0,2), (1,2), (2,2)
    ]
```

Its initial configuration is shown below. An **X** represents a live cell and a dot represents a dead cell.

```
.X.
..X
XXX
```

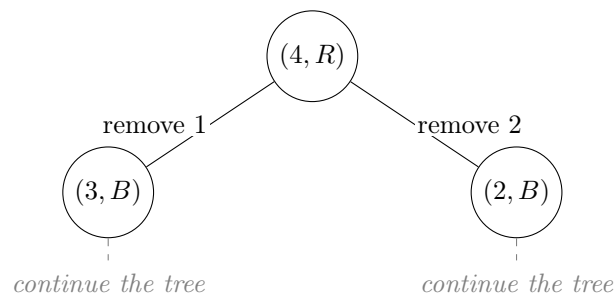
Find the exact outputs of the following expressions:

```
sort (runLife 1 glider)
sort (runLife 4 glider)
```

Briefly describe the relationship between the initial board and the board after four generations.

Q3 - A Simple Take-Away Game Tree

A pile initially contains four stones. Two players, Red and Blue, play alternately, and Red moves first. On each turn, a player must remove either one or two stones. A player may not remove more stones than remain in the pile. The player who removes the last stone wins. A game state could be represented by the pair (n, P) , where n is the number of stones remaining and P is the player whose turn it is. The game tree starts with $(4, R)$ and could be expanded until no stones remain. For example, the first level of the game tree is shown below.



- (a) The following Haskell data types can be used to represent the players and the game tree. The list stored in a **Node** contains all game states that can be reached in one legal move.

```
data Player = Red | Blue
    deriving (Eq, Show)

data GameTree = Node Int Player [GameTree]
    deriving (Eq, Show)
```

Complete the missing parts of the following definitions:

```
legalMoves :: Int -> [Int]
legalMoves n =
    [ m | m <- [1,2], _____ ]

buildGameTree :: Int -> Player -> GameTree
buildGameTree 0 p = Node 0 p []
buildGameTree n p =
    Node n p
        [ buildGameTree _____
          | m <- legalMoves n
          ]
```

The expression `buildGameTree 4 Red` must represent the same tree we partially drew.

- (b) Complete the following recursive functions. Use them to verify the node and leaf counts found in Part 4.

```
treeSize :: GameTree -> Int
treeSize (Node _ _ children) =
    1 + _____

leafCount :: GameTree -> Int
leafCount (Node _ _ []) = 1
leafCount (Node _ _ children) =
    _____
```

- (c) Every node could be labeled as either W or L from the perspective of the player whose turn it is:
- A terminal node is L, since the player whose turn it is cannot move.
 - A non-terminal node is W if at least one of its children is L.
 - A non-terminal node is L if all of its children are W.

Perform the labeling from the leaves toward the root.

- (d) How many nodes and how many terminal leaves are in the complete game tree?
- (e) Is the initial state $(4, R)$ a winning or losing position for Red? State which first move Red should choose and briefly justify your answer using the labels in the tree.

Q4 - List Comprehension

For each of the following Haskell expressions, write its exact result. Preserve the order of the elements in the output. If an expression continues evaluating forever and therefore produces no final result, write **NON-TERMINATING**.

- (a) `[x^2 | x <- [1..80], odd x, x `mod` 3 /= 0]`

- (b)

```
map (\x -> 2*x - 1) (filter (>3) [1,6,2,5,3,8])
```
- (c)

```
[(x,y) | x <- [1..4], y <- [x..4], z <- [y..4], even (x+y+z)]
```
- (d)

```
map (\xs -> sum (filter even xs))  
[[1,2,3], [4,5], [6,7,8]]
```
- (e)

```
[x-y | x <- [1,2,3], y <- [1..x]]
```
- (f)

```
filter (\xs -> xs /= [])  
  (map (filter odd) [[1,2], [2,4], [3,5], [6,7]])
```
- (g)

```
map length  
  [[y | y <- [1..x], even y] | x <- [1..5]]
```
- (h)

```
zipWith (+)  
  (map (*2) [1,2,3,4])  
  (filter odd [1..8])
```
- (i)

```
[x | x <- map (+1) [1..8], even x, x < 7]
```
- (j)

```
process :: [Int] -> [Int]  
process xs  
  | filter even xs == [] = xs  
  | otherwise = process (map (+2) xs)  
  
process [1,2,3]
```

Write the exact result of the function call, or write **Non-Terminating** if the call does not terminate. Briefly justify your answer.

Question 5 – Pancake Sort

A stack of pancakes is represented by a list of integers. The first element represents the pancake at the top of the stack. The only allowed operation is a *prefix flip*. A prefix flip reverses the first k elements of the list.

For example:

```
flipP 3 [5,2,4,1]  
-- [4,2,5,1]
```

Pancake Sort repeatedly performs the following operations:

- Find the largest element in the unsorted part of the stack.
- Flip the stack up to the position of the largest element, moving it to the top.
- Flip the entire unsorted part, moving the largest element to the bottom.
- Recursively sort the remaining elements.

Write the `pancakeSort` function in Haskell.

Question 6 – Constructor Patterns

Consider the following algebraic data types:

```
data Tree a
  = Nil
  | Node a (Tree a) (Tree a)
  deriving (Show, Eq)

data Expr
  = Val Int
  | Add Expr Expr
  | Neg Expr
  deriving (Show, Eq)
```

Complete only the missing **function parameter patterns**. Do not modify the right-hand sides of the equations.

- (a) Complete the function that returns `True` only when the root has two non-empty children.

```
hasTwoChildren :: Tree a -> Bool
hasTwoChildren
  ----- = True
hasTwoChildren _ = False
```

- (b) Complete the function that performs a left rotation when the right child of the root is non-empty. For every other tree, it returns the input unchanged.

```
rotateLeft :: Tree a -> Tree a
rotateLeft ----- =
  Node y (Node x left middle) right

rotateLeft tree = tree
```

- (c) Complete the function that recognizes a tree whose root has no left child, but whose right child is a leaf.

```
specialTree :: Tree a -> Bool
specialTree
  ----- = True
specialTree _ = False
```

- (d) Complete the following simplification rules. The first rule removes addition by zero, and the second rule removes two consecutive negations.

```
simplify :: Expr -> Expr
simplify ----- = simplify e

simplify ----- = simplify e

simplify (Add e1 e2) =
  Add (simplify e1) (simplify e2)

simplify (Neg e) =
  Neg (simplify e)

simplify e = e
```